

Hexabid Project Specification

Table of Contents

1. Project Vision	1
2. Architecture Overview	1
2.1. Hexagonal Architecture	1
2.2. Module Structure	2
3. Core Domain Models	2
3.1. Party (User)	2
3.2. Auction	2
3.3. Lot (Item for Sale)	3
3.4. Product	3
4. Business Requirements	3
4.1. User Registration and Profile	3
4.2. Auction Creation	3
4.3. Real-Time Bidding	3
4.4. Payment Processing	3
4.5. Event Publishing	4
5. API Specifications	4
5.1. REST Endpoints	4
5.2. WebSocket Channels	4
5.3. Authentication	4
6. Non-Functional Requirements	4
7. Development Principles	4

1. Project Vision

Hexabid is a reference auction platform implementing **hexagonal architecture** (ports and adapters) with **contract-first** development approach. The system demonstrates enterprise-grade patterns for real-time bidding, payment processing, and clean domain separation.

2. Architecture Overview

2.1. Hexagonal Architecture

The system follows the hexagonal (ports and adapters) architecture pattern:

- **Core Domain** (**auctions-core**) - Pure Java business logic, no framework dependencies
- **Input Adapters** - REST, WebSocket, Job scheduler (inbound)

- **Output Adapters** - JPA, Kafka, KYC, Payment gateways (outbound)
- **Ports** - Interfaces defining boundaries between core and adapters

2.2. Module Structure

```

quantity/                - Unit and Quantity value objects
product/                 - Product catalog domain
inventory/              - Inventory management domain
auctions-core/          - Core auctioning domain (Pure Java)
auctions-auth-core/     - Authentication domain (Party model)
hexabid-api-contract/   - OpenAPI contract definitions
auctions-adapter-in-rest/ - REST API adapter
auctions-adapter-in-ws/  - WebSocket STOMP adapter
auctions-adapter-in-job/ - Scheduler adapter
auctions-adapter-in-auth-oauth/ - OAuth2/OpenID Connect adapter
auctions-adapter-in-auth-local/ - Local form login adapter (dev)
auctions-adapter-out-db/ - JPA persistence adapter
auctions-adapter-out-kafka/ - Event publishing adapter
auctions-adapter-out-kyc/ - KYC verification client
auctions-adapter-out-kyc-local/ - Local KYC mock (dev)
auctions-payment-core/  - Payment processing domain
auctions-payment-api-contract/ - Payment contract
auctions-payment-adapter-payu/ - PayU gateway
auctions-payment-adapter-p24/ - Przelewy24 gateway
auctions-payment-adapter-crypto/ - Crypto payments
auctions-payment-adapter-local/ - Local mock (dev)
hexabid-bootstrap/      - Spring Boot composition root

```

3. Core Domain Models

3.1. Party (User)

- **Party** - Business participant (buyer/seller) independent of authentication mechanism
- **PartyId** - Unique identifier derived from identity provider
- **PartyRole** - Seller, Buyer, Admin

3.2. Auction

- **Auction** - Aggregate root with bidding rules
- **AuctionId** - Unique auction identifier

- **AuctionStatus** - OPEN, CLOSED
- **Bid** - Individual bid with amount, currency, timestamp

3.3. Lot (Item for Sale)

- **Lot** - Represents item being auctioned
- **LotId** - Unique lot identifier
- **SellingMode** - WHOLE, DIVISIBLE, DIVISIBLE_ONLY

3.4. Product

- **Product** - Product definition in catalog
- **ProductId** - Unique product identifier

4. Business Requirements

4.1. User Registration and Profile

- Users authenticate via OAuth2/OIDC (Google, GitHub) or local credentials
- Profile includes identity verification status (KYC)

4.2. Auction Creation

- Seller creates auction with title, description, starting price, end time
- KYC verification required before creating auctions
- Real-time validation of business rules

4.3. Real-Time Bidding

- WebSocket/STOMP for live bidding
- Optimistic locking for concurrent bid handling
- Leader change notifications
- Business result instead of exceptions

4.4. Payment Processing

- Multiple payment gateways (PayU, Przelewy24, Crypto)
- Currency conversion support
- Accounting entries per transaction

4.5. Event Publishing

- Domain events to Kafka
- WebSocket notifications to clients
- Event types: AuctionWon, AuctionClosedWithoutWinner, AuctionLeaderChanged

5. API Specifications

5.1. REST Endpoints

- `POST /api/auctions` - Create auction
- `GET /api/auctions/{id}` - Get auction details
- `GET /api/auctions` - Browse/search auctions

5.2. WebSocket Channels

- `/app/auctions/{id}/bids` - Place bid
- `/topic/auctions/{id}/bids` - Accepted bids
- `/topic/auctions/{id}/events` - Domain events
- `/topic/auctions/{id}/errors` - Business rejections

5.3. Authentication

- OAuth2/OpenID Connect with Google and GitHub
- Local form login for development

6. Non-Functional Requirements

- **Scalability** - Handle concurrent bidding with optimistic locking
- **Security** - OAuth2/OIDC authentication, no credential storage
- **Modularity** - Independent modules with clear boundaries
- **Contract-First** - OpenAPI definitions before implementation

7. Development Principles

- Domain logic in pure Java (no Spring/JPA in core)
- Explicit business results instead of exceptions leaking to adapters
- Contract-first with OpenAPI generator
- Event-driven with Kafka and WebSocket

- Plugin architecture for authentication (profile-based)